Contexto

Una plataforma de reseñas de cine quiere entender qué relación existe entre lo que sus usuarios escriben sobre una película y el puntaje que finalmente le asignan.

Para ello cuentan con un dataset de aproximadamente 90.000 reseñas de **Letterboxd**, que incluye tanto el **texto de la reseña** como el **puntaje en estrellas**, la fecha, la película reseñada y un campo de estado.

El objetivo es construir un modelo que pueda **predecir el puntaje de una reseña (Rating)** a partir de su contenido y de la información disponible. Esto les permitiría completar el puntaje de reseñas que no lo tienen, detectar inconsistencias entre el texto y la calificación, y ordenar automáticamente nuevas reseñas a medida que se recolectan.

Columnas disponibles: **Review, Rating, Date, Status, Movie**.

Desafío

El desafío consiste en desarrollar un **pipeline completo de Machine Learning** que permita predecir el puntaje de una reseña a partir de su texto.

Esto implica realizar todas las etapas típicas de un proyecto de ciencia de datos:

- Exploración del dataset
- Análisis de las variables disponibles
- Preprocesamiento del texto
- Ingeniería de features
- Entrenamiento de modelos
- Evaluación de resultados

El objetivo no es únicamente entrenar un modelo, sino **analizar las decisiones tomadas en cada etapa del proceso**.

Presten especial atención a una característica de este problema: el target no es una variable categórica cualquiera, sino una variable **ordinal** (de 0.5 a 5 estrellas, de a media estrella). Decidir cómo tratar esa estructura es una parte central del trabajo.

Lineamientos

Durante el desarrollo del trabajo deberán investigar y justificar decisiones en distintas partes del pipeline. Algunas preguntas que pueden servir como punto de partida:

Exploración del dataset

Analicen la columna **Rating**.

- ¿Qué valores toma exactamente? ¿Cuántas clases distintas hay?
- ¿Cuántas reseñas hay por cada valor? ¿El dataset está balanceado?
- ¿Se observa un sesgo hacia los puntajes altos? ¿Las medias estrellas aparecen con la misma frecuencia que las estrellas enteras?
- ¿Cómo puede afectar esto al entrenamiento de los modelos y a la métrica que elijan?
- ¿Hay reseñas sin puntaje? ¿Qué harían con ellas?

Realicen un análisis exploratorio inicial del dataset:

- ¿Cuántas reseñas contiene? ¿Cuántas películas distintas?
- ¿Qué columnas están disponibles?
- ¿Qué tipos de datos tiene cada variable?
- ¿Existen valores faltantes? ¿Y reseñas duplicadas?
- ¿Cuál es la longitud típica de una reseña? ¿Hay reseñas vacías o de una sola palabra?

Análisis de las variables disponibles

Además del texto, el dataset incluye **Movie**, **Date** y **Status**. Explórenlas:

- **Movie:** ¿cuántas reseñas hay por película? ¿El puntaje promedio varía mucho de una película a otra?
- Si usaran la película como feature, ¿el modelo seguiría siendo útil para una película que no aparece en el dataset? Discutan las ventajas y los riesgos.
- **Date:** ¿el puntaje promedio cambia a lo largo del tiempo? ¿Cambia la longitud de las reseñas?
- **Status:** investiguen qué información contiene esta columna y si aporta algo al problema.
- ¿Se observan outliers en la longitud de las reseñas? ¿Qué distribución tiene esa variable?

Piensen también cómo van a **separar train y test**: ¿un split aleatorio, uno temporal, o uno que garantice que las reseñas de una misma película no queden repartidas entre ambos conjuntos? Justifiquen qué mide cada alternativa.

Ingeniería de features

Además de las variables existentes, pueden generar **nuevas features** que ayuden al modelo. Por ejemplo:

- Longitud de la reseña (en caracteres, palabras u oraciones).
- Cantidad de signos de exclamación o de interrogación.
- Proporción de palabras escritas en mayúsculas.
- Cantidad de emojis.
- Antigüedad de la reseña (año o mes extraído de Date).

Justifiquen por qué creen que estas variables podrían ser útiles.

Analicen también cómo **combinar** estas features numéricas con la representación vectorial del texto, y si hace falta escalarlas antes de entrenar.

Preprocesamiento de texto

Antes de aplicar técnicas de NLP como **Bag of Words** o **TF-IDF**, analicen cómo tratar distintos elementos del texto:

- Mayúsculas y puntuación (¿es lo mismo “amazing” que “AMAZING!!!”?).
- Emojis.
- Negaciones: si eliminan stopwords, “not good” puede terminar convertido en “good”.
- Nombres propios: títulos de películas, directores, actores.
- Reseñas muy cortas, vacías o escritas en otro idioma.

Reflexionen sobre si estos elementos deberían eliminarse, si podrían transformarse, si podrían construirse features a partir de ellos o si conviene mantenerlos tal cual están.

Experimenten además con stemming y lematización, con el uso de **n-gramas** (unigramas vs. bigramas) y con los parámetros del vectorizador (*min_df*, *max_df*, *max_features*). Observen cómo impactan en la cantidad de features resultantes.

Modelado

Entrenen distintos modelos. Pueden utilizar tanto modelos vistos en clase como otros disponibles en **Scikit-Learn**. Por ejemplo:

- Naive Bayes
- Logistic Regression
- Decision Tree
- Random Forest
- Support Vector Machines

Como el target es ordinal, comparen al menos dos formas de plantear el problema: como **clasificación multiclase** (una clase por cada valor de estrellas) y como **regresión** sobre el puntaje, redondeando después la predicción a la media estrella más cercana. ¿Cuál funciona mejor y por qué?

Investiguen qué **hiperparámetros pueden ajustarse** en cada modelo y experimenten con diferentes configuraciones. Recuerden lo discutido en clase sobre cómo **SVM escala con la cantidad de features**, especialmente en representaciones de texto. Con 90.000 reseñas los tiempos de entrenamiento pueden volverse un problema: consideren trabajar sobre una muestra mientras exploran, y entrenar sobre el total recién al final.

Definan también un **baseline**: ¿qué resultado obtiene un modelo que siempre predice la clase mayoritaria? Todo lo que entrenen debería compararse contra ese piso.

Evaluación

Definan qué **métrica quieren optimizar** para este problema y justifíquelo. Tengan en cuenta que se trata de un problema **multiclase y ordinal**: confundir 4.5 con 5 estrellas no es un error tan grave como confundir 0.5 con 5, pero el accuracy trata a los dos errores exactamente igual.

- ¿Qué aportan métricas como MAE o RMSE en este contexto?
- ¿Tiene sentido medir el acierto con una tolerancia de media estrella o de una estrella?
- Analicen la matriz de confusión: ¿los errores se concentran cerca de la diagonal?
- ¿Hay clases que el modelo nunca predice? ¿Qué relación tiene eso con el desbalance?

Analicen los resultados de sus modelos y comparen su desempeño entre sí y contra el baseline.

Entregables

Cada grupo deberá entregar:

Un archivo .ipynb que contenga:

- El análisis exploratorio.
- El preprocesamiento del texto.
- La ingeniería de features.
- El entrenamiento de modelos.
- La evaluación de resultados.
- Conclusiones.

El notebook debe incluir **explicaciones claras de las decisiones tomadas**.

Una breve presentación de 20 minutos donde expliquen:

- Cómo abordaron el problema.
- Hallazgos encontrados al analizar el Dataset.
- Qué decisiones tomaron (features, limpieza, etc).
- Qué funcionó mejor (y qué probaron).
- Qué mejoras implementarían en el futuro.

Importante

- **El notebook debe poder ejecutarse completamente utilizando Run All.**
- Pueden usar herramientas de IA generativa durante el desarrollo del trabajo, siempre que el uso quede explicitado. Incluyan al final del notebook una breve sección indicando en qué partes del proceso las usaron y con qué fin (por ejemplo: entender un mensaje de error, explorar alternativas de preprocesamiento, redactar conclusiones). Lo que se evalúa en este trabajo son las decisiones que toman y la justificación que dan de ellas, no la cantidad de código producido: una solución generada automáticamente que no puedan explicar vale menos que una más simple que entiendan por completo. **Tengan en cuenta que durante la presentación se les van a hacer preguntas sobre cualquier parte del notebook, incluidas las que no hayan escrito ustedes.**